

Complete Computer Science Study Guide

AI & ML | DBMS | Computer Networks

Comprehensive reference from fundamentals to advanced topics.
Covers Supervised Learning, Deep Learning, Transformers, LLMs,
RL / GRPO, Database Design, Normalization, OSI Model,
Subnetting, Protocols, and more.

Generated for interview and exam preparation.

PART I: Artificial Intelligence & Machine Learning

1. What is Machine Learning?

1.1 Core Idea

Traditional programming: you write rules → computer applies rules to data → output.

Machine Learning: you give data + outputs → computer *finds the rules itself*.

The "rules" are parameters (weights) inside a model. Training = adjusting weights until model output matches desired output.

1.2 Three Learning Paradigms

Paradigm	Description	Examples
Supervised Learning	Labeled data. Learn mapping $f(x) \rightarrow y$.	Image classification, spam detection, MCQ generation
Unsupervised Learning	No labels. Find structure in raw data.	Clustering (k-means), PCA, autoencoders
Reinforcement Learning	Agent takes actions, gets rewards. Learn policy: state → action maximizing cumulative reward.	AlphaGo, ChatGPT alignment (RLHF), game AI

2. Math Foundations

2.1 Tensors

- **Scalar:** single number. $x = 3.14$
- **Vector:** 1D array. $x = [1, 2, 3]$, shape (3,)
- **Matrix:** 2D array. Shape (rows, cols) e.g. (512, 768)
- **Tensor:** N-dimensional generalization. Batch of images: shape (batch, channels, H, W) = 4D tensor

2.2 Dot Product

$$a \cdot b = \sum a_i b_i = |a| |b| \cos(\theta)$$

Measures alignment between vectors. If $a \cdot b = 0$, orthogonal (unrelated). If high, similar direction. Core of attention mechanisms.

2.3 Activation Functions

Add **nonlinearity**. Without them, stacking linear layers = still just one linear layer. Useless for learning complex patterns.

Function	Formula	Range	Use / Notes
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	(0, 1)	Binary classification output. Problem: vanishing gradient at extremes.
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	(-1, 1)	Better than sigmoid but still saturates.
ReLU	$\max(0, x)$	$[0, \infty)$	Default hidden layers. Fast. Problem: dying ReLU if always negative input.
LeakyReLU	$\max(0.01x, x)$	$(-\infty, \infty)$	Fixes dying ReLU with small negative slope.
GELU	$x \times \Phi(x)$	$(-\infty, \infty)$	Used in GPT, BERT. Smooth. Empirically best for transformers.
SiLU/Swish	$x \times \sigma(x)$	$(-\infty, \infty)$	Used in LLaMA, Mistral.
SoftMax	$e^{x_i} / \sum e^{x_j}$	(0,1), sums to 1	Multi-class output. Also in attention.

2.4 Loss Functions

Loss = how wrong the model is. Training = minimize loss.

Loss	Formula	Use
MSE	$(1/n) \sum (\hat{y} - y)^2$	Regression
Cross-Entropy	$-\sum y \times \log(\hat{y})$	Multi-class classification
Binary Cross-Entropy	$-[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$	Binary classification

3. Forward Propagation

Data flows forward through layers producing prediction.

3.1 Single Neuron

```
output = activation(W · x + b)
```

W = weights, b = bias, x = input.

3.2 Multi-Layer Perceptron (MLP)

```
Layer 1: h1 = activation(W1 · x + b1)
Layer 2: h2 = activation(W2 · h1 + b2)
Layer 3: y_hat = softmax(W3 · h2 + b3)
```

Each layer applies learned linear transformation then nonlinearity.

- **Early layers:** simple patterns (edges, common tokens)
- **Middle layers:** combinations (shapes, phrases)
- **Late layers:** high-level concepts (cat vs dog, topic classification)

4. Backward Propagation

4.1 The Problem

We have loss L. We need: how does each weight w affect L? Use **gradient** = $\partial L / \partial w$. Gradient tells direction of steepest increase. Go opposite direction to reduce loss.

4.2 Chain Rule

$$\partial L / \partial w = (\partial L / \partial h) \times (\partial h / \partial w)$$

For multi-layer network:

$$\partial L / \partial w_1 = (\partial L / \partial \hat{y}) \times (\partial \hat{y} / \partial h_2) \times (\partial h_2 / \partial h_1) \times (\partial h_1 / \partial w_1)$$

Error propagates backward from output → input. Hence "backpropagation."

4.3 Steps

1. Forward pass: compute all layer outputs, store them
2. Compute loss $L = \text{CrossEntropy}(\hat{y}, y)$
3. Compute $\partial L / \partial \hat{y}$ — gradient at output
4. Walk backward applying chain rule at each layer

5. Accumulate gradients for every W and b

4.4 Vanishing Gradient Problem

Chain rule multiplies gradients. Sigmoid/tanh saturate $\rightarrow$ derivative $\rightarrow 0$. Multiply many near-zeros $\rightarrow$ gradient at early layers ≈ 0 . Early layers stop learning.

Solutions: ReLU, residual connections (skip connections), batch normalization, Xavier/He initialization.

4.5 Exploding Gradient Problem

Gradients grow exponentially $\rightarrow$ NaN instability.

Solution: gradient clipping: if $\|\text{grad}\| > \text{threshold}$, scale down.

5. Gradient Descent — All Variants

5.1 Core Update Rule

$$w \leftarrow w - \eta \times \partial L / \partial w$$

η (eta) = learning rate. Size of each step.

Variant	Batch Size	Pros	Cons
Batch GD	Full dataset	Stable gradient	Impractical for large datasets
SGD	1 sample	Fast, online	High variance, noisy
Mini-Batch GD	32–2048	Efficient GPU use + reasonable variance	Needs LR tuning

5.2 Momentum

$$v \leftarrow \beta \times v + (1-\beta) \times \text{grad } w \leftarrow w - \eta \times v \quad (\beta \approx 0.9)$$

Builds speed in consistent gradient direction, dampens oscillations. Like a ball rolling downhill with inertia.

5.3 RMSProp

$$E[g^2] \leftarrow \beta \times E[g^2] + (1-\beta) \times \text{grad}^2 \quad w \leftarrow w - (\eta / \sqrt{E[g^2] + \epsilon}) \times \text{grad}$$

Adaptive per-parameter learning rates. Parameters with large gradient history get smaller effective LR.

5.4 Adam (Adaptive Moment Estimation)

Adam = Momentum + RMSProp. Maintains first moment m (mean of gradients) and second moment v (mean of squared gradients).

$$m \leftarrow \beta_1 \times m + (1-\beta_1) \times \text{grad} \quad (\beta_1 = 0.9) \quad v \leftarrow \beta_2 \times v + (1-\beta_2) \times \text{grad}^2 \quad (\beta_2 = 0.999) \quad \hat{m} = m / (1-\beta_1^t) \quad (\text{bias correction}) \quad \hat{v} = v / (1-\beta_2^t) \quad w \leftarrow w - \eta \times \hat{m} / (\sqrt{\hat{v}} + \epsilon) \quad (\epsilon = 1e-8)$$

AdamW: Adam + correct weight decay (applied directly to weights, not via gradient). Used in GPT, BERT, LLaMA.

5.5 Learning Rate Schedulers

- **Warmup:** start tiny, ramp up. Prevents early instability.
- **Cosine decay:** LR follows cosine curve peak $\rightarrow$ near-0. Standard in LLM training.
- **Linear decay / Step decay:** alternatives.

6. Regularization

Technique	How	Effect
L2 / Weight Decay	Add $\lambda \Sigma w^2$ to loss	Pushes weights toward 0, prevents over-reliance on any feature
Dropout	Randomly zero neurons with prob p during training	Forces redundancy, acts like ensemble of sub-networks
Batch Normalization	Normalize activations within batch, learnable scale+shift	Stable training, less sensitive to init
Layer Normalization	Normalize across features of single example	Used in transformers (batch norm fails on variable-length sequences)

7. Convolutional Neural Networks (CNNs)

Specialized for grid data (images, 1D signals).

- **Convolution:** slide small filter across input, compute dot product. Learns local patterns.
- **Weight sharing:** same kernel everywhere $\rightarrow$ huge parameter reduction vs fully-connected.
- **Translation invariance:** pattern detected regardless of position.
- **Pooling:** downsample feature maps. Max pool = max in each region.

Input image $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ Softmax

8. Recurrent Neural Networks (RNNs)

For sequential data. At each timestep, takes input + previous hidden state:

$$h_t = \tanh(Wx \times x_t + Wh \times h_{t-1} + b)$$

Problem: vanishing gradient over long sequences (BPTT multiplies many Wh terms). Standard RNNs forget long-range context.

8.1 LSTM (Long Short-Term Memory)

Adds cell state C — a separate memory highway with gated updates:

- **Forget gate:** $f = \sigma(W_f \times [h, x] + b)$ — how much old memory to keep
- **Input gate:** $i = \sigma(W_i \times [h, x] + b)$ — what new info to write
- **Cell update:** $C = f \odot C_{\text{prev}} + i \odot \tanh(W_c \times [h, x] + b)$
- **Output gate:** what to expose as hidden state

Solves vanishing gradient — cell state gradients can flow unchanged through forget gate (when $f \approx 1$).

8.2 GRU (Gated Recurrent Unit)

~~Simplified LSTM — fewer gates, fewer parameters, similar performance. Both largely obsolete for NLP — transformers beat them decisively by 2019.~~

9. Attention Mechanism

9.1 Problem with RNNs

Encoder compresses entire input into single fixed-size vector. Long sequences → information bottleneck. Word meaning depends on distant context ("bank" meaning depends on "river" 10 words away).

9.2 Basic Attention

$$\text{score}(h_i, s) = h_i \cdot s \quad \alpha_i = \text{softmax}(\text{scores}) \quad \text{context} = \sum \alpha_i \times h_i$$

Decoder attends to whichever encoder positions are most relevant. Dramatically improved translation (Bahdanau et al., 2014).

9.3 Self-Attention

Each position attends to all other positions in the same sequence. Captures dependencies regardless of

distance.

1. Project each input x_i into Query, Key, Value:

$$Q_i = W_q \times x_i, K_i = W_k \times x_i, V_i = W_v \times x_i$$

2. Score each pair: $\text{score}(i, j) = Q_i \cdot K_j / \sqrt{d_k}$ (scaling prevents large dot products $\rightarrow$ softmax saturation)
3. Attention weights: $\alpha_{ij} = \text{softmax}(\text{scores}[i, :])$
4. Output: $o_i = \sum_j \alpha_{ij} \times V_j$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

Interpretation: Q = "what am I looking for", K = "what do I contain", V = "what I will contribute." High Q·K score = relevant match.

9.4 Multi-Head Attention

$$\text{head}_i = \text{Attention}(QW_{iq}, KW_{ik}, VW_{iv}) \quad \text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \times W_o$$

Different heads learn different attention patterns: syntax, coreference, topic, etc.

10. The Transformer Architecture

Vaswani et al., "Attention Is All You Need" (2017). No recurrence, no convolution — pure attention.

10.1 Encoder Block (BERT-style)

Input $\rightarrow$ Embedding + Positional Encoding
 $\rightarrow$ [Multi-Head Self-Attention $\rightarrow$ Add & Norm] $\times N$
 $\rightarrow$ [Feed-Forward (2-layer MLP) $\rightarrow$ Add & Norm] $\times N$
 $\rightarrow$ Output representations

10.2 Decoder Block (GPT-style)

Input $\rightarrow$ Embedding + Positional Encoding
 $\rightarrow$ [Masked Self-Attention $\rightarrow$ Add & Norm] $\times N$ (causal mask)
 $\rightarrow$ [Cross-Attention $\rightarrow$ Add & Norm] $\times N$ (encoder-decoder only)
 $\rightarrow$ [Feed-Forward $\rightarrow$ Add & Norm] $\times N$
 $\rightarrow$ Linear $\rightarrow$ Softmax $\rightarrow$ Next token probabilities

10.3 Residual Connections ("Add & Norm")

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Highway for gradients. Critical for training deep models (100+ layers). Without this, gradients can't flow back far enough.

10.4 Positional Encoding

Self-attention is order-invariant — treats input as a set. Position must be injected explicitly.

Method	Formula / Idea	Used In
Sinusoidal	$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$	Original Transformer
Learned absolute	Embedding table indexed by position	GPT-2, BERT
RoPE	Rotate Q and K in complex space by position angle	LLaMA, Mistral, Gemma
ALiBi	Linear penalty added to attention scores based on distance	Some LLaMA variants

10.5 Causal Masking

LLM predicts next token — at position i, must not see tokens i+1, i+2... Apply mask: $\text{score}(i,j) = -\infty$ if $j > i$.
After softmax, $-\infty \rightarrow 0$ attention weight.

--

10.6 Feed-Forward Network (FFN)

$$FFN(x) = \max(0, xW1 + b1)W2 + b2$$

Two linear layers with ReLU. Typically 4x wider than model dim ($d=768 \rightarrow FFN=3072$). Attention routes/ mixes information between positions. FFN stores factual knowledge and reasoning patterns.

SwiGLU (LLaMA variant):

$$FFN(x) = (\text{SiLU}(xW1) \odot xW2) W3$$

Gated FFN. Better than ReLU FFN empirically. Used in LLaMA 2/3, Mistral.

--

11. Encoder vs Decoder vs Encoder-Decoder

Architecture	Examples	Use Case	Key Property
Encoder-only	BERT, RoBERTa	Classification, NER, semantic search	Bidirectional — sees full context both ways
Decoder-only	GPT, LLaMA, Mistral, Gemini	Chat, completion, generation	Causal/unidirectional

Encoder-Decoder	T5, BART, mT5	Translation, summarization, Q-generation	Full context encoding + autoregressive decoding
-----------------	---------------	--	---

Why decoder-only dominates for LLMs: Simpler, scales better, emergent few-shot abilities, can do any task via prompting.

12. Tokenization

Method	Unit	Pros	Cons
Word	Whole words	Human-readable tokens	OOV problem for rare words
Character	Single char	No OOV	Very long sequences, context wasted
BPE	Subword	Handles rare words, compact	Numbers tokenized badly
WordPiece	Subword	Likelihood-based merges	Same as BPE
SentencePiece	Byte-level subword	Language-agnostic (Devanagari etc.)	Slightly larger vocab

BPE (Byte Pair Encoding): start with characters, repeatedly merge most frequent adjacent pair into new token until vocab size reached (32k–128k). Used in GPT, LLaMA.

Why tokenization matters: Context window = token limit. Numbers split weirdly → arithmetic hard. "dog" ≠ " dog" with leading space in many tokenizers.

13. LLM Training

13.1 Data

LLaMA 2: 2T tokens. GPT-4: estimated 10T+ tokens. Sources: Common Crawl, Wikipedia, books, code (GitHub), academic papers, StackExchange.

Data quality pipeline: deduplication (MinHash) → language filtering → quality filtering (perplexity) → toxic/NSFW removal → PII removal.

13.2 Pre-training Objective

CLM (Causal Language Modeling): predict next token. Self-supervised — no human labels needed.

Loss = $-\sum_t \log P(x_t \mid x_1, \dots, x_{t-1})$

Given sequence ["The", "cat", "sat"], model predicts $P(\text{"cat"}|\text{"The"})$, $P(\text{"sat"}|\text{"The","cat"})$, etc. Backprop

through cross-entropy → model learns grammar, facts, reasoning, style.

MLM (BERT): mask 15% of tokens, predict masked tokens. Bidirectional. Can't do generation directly.

13.3 Distributed Training

Strategy	How	When to Use
Data Parallelism	Copy model to N GPUs, each processes different batch, gradients averaged	Model fits on one GPU
Tensor Parallelism	Split weight matrices across GPUs within one layer	Very large models, fast interconnect
Pipeline Parallelism	Different layers on different GPUs	Too many layers for one GPU
ZeRO / FSDP	Shard optimizer states, gradients, params across GPUs	Largest models, memory bottleneck

13.4 Mixed Precision

Train in BF16 (2 bytes/param, wide exponent range). Maintain FP32 master copy for precision. 2× memory reduction, 2-4× faster compute.

14. Key Transformer Design Choices

Feature	GPT-2 (2019)	LLaMA 2 (2023)	LLaMA 3 (2024)
Attention	Multi-Head	Grouped Query (GQA)	Grouped Query (GQA)
FFN	ReLU	SwiGLU	SwiGLU
Normalization	Post-LayerNorm	RMSNorm (Pre-LN)	RMSNorm (Pre-LN)
Position	Learned absolute	RoPE	RoPE
Vocab size	50k	32k	128k
Context	1024	4096	8192

14.1 GQA (Grouped Query Attention)

Multi-Head: N query heads, N key heads, N value heads. Multi-Query (MQA): N query, 1 shared KV — fast but quality loss. **GQA**: N query, G groups sharing KV ($G < N$). Balance. LLaMA 2 70B: 64 Q heads, 8 KV heads.

Why KV cache matters: At each generation step, KV for previous tokens is cached (reuse). For long

sequences KV cache = GB of VRAM. GQA reduces this by N/G ratio.

14.2 RMSNorm

$$\text{RMSNorm}(x) = x / \text{RMS}(x) \times \gamma, \quad \text{RMS}(x) = \sqrt{\text{mean}(x^2)}$$

Simpler than LayerNorm (no mean subtraction). ~7% faster. Applied before sublayer (Pre-LN) for training stability.

15. Fine-Tuning

15.1 Full Fine-Tuning

Update all parameters on task-specific data. Most powerful. Most expensive. Risk of catastrophic forgetting.

15.2 LoRA (Low-Rank Adaptation)

Key insight: weight updates during fine-tuning are low-rank — most singular values near zero.

$$W' = W + \Delta W = W + BA$$

Where B is (d×r) and A is (r×k) with rank $r \ll \min(d, k)$. If W is 4096×4096 = 16.7M params and r=16: A+B = 130k params. ~128× reduction.

- **QLoRA:** quantize base model to 4-bit (NF4), train LoRA adapters in BF16. Fine-tune 70B on single A100.
- **DoRA:** decompose W into magnitude + direction, apply LoRA to direction. Better quality.
- **rsLoRA:** scale LoRA by $1/\sqrt{r}$. More stable.

15.3 Instruction Fine-Tuning (SFT)

Train on (instruction, response) pairs. Transforms base model into instruction follower. Small high-quality data (1k–100k examples) sufficient.

```
[INST] What is photosynthesis? [/INST]
Photosynthesis is the process by which plants...
```

16. RLHF — Reinforcement Learning from Human Feedback

16.1 Step 1: SFT

Train base model on instruction data → SFT model.

16.2 Step 2: Reward Model Training

Collect comparison data: show human two responses, human picks preferred. Train Reward Model (RM) that assigns scalar score to (prompt, response) pairs.

$$L_{RM} = -\log \sigma(r(x, y_w) - r(x, y_l))$$

y_w = preferred response, y_l = rejected. RM learns what humans prefer.

16.3 Step 3: PPO Optimization

Fine-tune SFT model (policy π) to maximize RM reward with KL divergence penalty:

$$\text{objective} = E[r(x, y)] - \beta \times KL(\pi \parallel \pi_{SFT})$$

KL penalty prevents reward hacking and catastrophic forgetting of SFT.

PPO (Proximal Policy Optimization):

$$L_{PPO} = E[\min(\text{ratio} \times \text{advantage}, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \times \text{advantage})] \quad \text{ratio} = \pi(a|s) / \pi_{old}(a|s)$$

Clips ratio to $[1-\epsilon, 1+\epsilon]$. Prevents large destabilizing updates.

PPO problems for LLMs: needs 4 models simultaneously (policy, reference, reward model, critic) — huge memory. Unstable. Slow.

17. DPO — Direct Preference Optimization

Stanford 2023. Eliminates reward model + PPO entirely.

$$L_{DPO} = -E[\log \sigma(\beta \times \log(\pi(y_w|x)/\pi_{ref}(y_w|x)) - \beta \times \log(\pi(y_l|x)/\pi_{ref}(y_l|x)))]$$

Directly maximize probability of preferred over rejected response, relative to reference model. No separate RM. No PPO loop. Only 2 models (policy + reference). Stable, works with LoRA.

Variants: IPO (avoids DPO failure modes), KTO (works on unpaired win/loss data), SimPO (reference-model free).

18. GRPO — Group Relative Policy Optimization

DeepSeek AI, 2024. Used in DeepSeek-R1. Alternative to PPO that eliminates the critic (value function).

18.1 Core Idea

For each question x , sample G outputs. Compute reward r_i for each. Estimate baseline from group statistics:

$$\text{Advantage}_i = (r_i - \text{mean}(r)) / \text{std}(r)$$

Positive advantage → reinforce. Negative → suppress. No critic model needed

18.2 Loss

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}[\sum_i (\min(\text{ratio} \times A_i, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \times A_i)) - \beta \times \text{KL}]$$

18.3 Why GRPO for Reasoning

- Uses verifiable rewards (math: correct/incorrect — no learned RM needed)
- Group comparison naturally captures relative quality of reasoning chains
- No critic → 50% fewer model copies during training
- Excellent for math, code, logic where rewards are objective

18.4 Outcome vs Process Reward

Type	Reward Signal	Data Requirement
ORM (Outcome)	Final answer correct/wrong	Easy — just check answer
PRM (Process)	Each reasoning step valid	Hard — need step-level labels

DeepSeek-R1 uses both. PRM improves intermediate reasoning quality.

19. Chain-of-Thought and Reasoning Models

- **CoT (Chain-of-Thought):** "think step by step" before answering. Huge accuracy gains on math/reasoning.
- **Few-Shot CoT:** show examples with step-by-step reasoning in prompt.
- **Zero-Shot CoT:** "Let's think step by step." Works at scale without examples.
- **o1/R1 style:** model trained to produce long CoT before final answer. RL signal only on correctness of final answer. Model learns to search, self-verify, backtrack. More thinking tokens → better answers (test-time compute scaling).

20. Quantization

Format	Bytes/param	70B Model Size	Quality
FP32	4	280 GB	Reference
BF16/FP16	2	140 GB	Near-lossless
INT8 (Q8)	1	70 GB	Nearly lossless
Q6_K	0.75	~52 GB	Excellent
Q4_K_M	0.5	~35 GB	Good balance (popular)
Q3_K_M	0.375	~26 GB	Visible quality loss
Q2_K	0.25	~18 GB	Significant degradation

GPTQ: uses inverse Hessian information to minimize quantization error. **AWQ**: activation-aware — preserves important weights (large activation magnitude). **GGUF**: single-file format with weights + tokenizer + architecture. Supports mixed quantization. Standard for local inference (llama.cpp).

21. llama.cpp

Georgi Gerganov, 2023. C/C++ inference engine. No Python, no CUDA required (though CUDA/Metal supported).

- Loads GGUF model file
- CPU inference with AVX2/AVX512 SIMD
- GPU offloading: N layers on GPU, rest on CPU
- Flash attention, continuous batching
- Metal backend (Apple Silicon), CUDA backend (NVIDIA)
- Powers: Ollama, LM Studio, Jan

Inference loop:

```
For each token to generate:
  1. Load KV cache for all previous tokens
  2. Forward pass through N transformer layers
  3. Compute logits for vocabulary
  4. Sample next token (temperature, top-p, top-k)
  5. Append to KV cache
  6. Repeat until [EOS] or max_tokens
```

22. vLLM

22. PagedAttention (Continuous batching)

KV cache stored in non-contiguous blocks (pages, ~16 tokens each). Physical blocks allocated on demand. Logical sequences mapped to physical blocks via block table. Near-zero KV cache waste (<4% fragmentation). Can batch 20-40× more requests.

Features

- **Continuous batching:** dynamically add new requests as old ones complete. Maximizes GPU utilization
- **Token streaming:** send tokens to client as generated.
- **Throughput:** 2-24× higher than naive HuggingFace inference.
- **Alternatives:** TensorRT-LLM (NVIDIA, fastest, NVIDIA-only), TGI (HuggingFace)

23. Sampling Strategies

Strategy	How	Effect
Greedy	Always pick argmax	Deterministic, often repetitive
Temperature	$\text{softmax}(\text{logits} / T)$. $T < 1$ sharpens, $T > 1$ flattens	Controls creativity/randomness
Top-K	Sample from top K tokens only	Eliminates very unlikely tokens
Top-P (nucleus)	Smallest set where cumulative prob $\geq p$	Adaptive — tight when model confident
Min-P	Keep tokens with $\text{prob} \geq \text{min_p} \times \text{max_token_prob}$	Relative threshold, clean outputs
Repetition penalty	Divide logit of recent tokens by $\text{penalty} > 1$	Reduces loops and repetition

Common default: temperature=0.7, top_p=0.9.

24. Context Window and KV Cache

Context window: max tokens processable at once. GPT-4: 128k. LLaMA 3: 8k base, 128k extended.

KV cache size formula:

$$2 \times n_layers \times n_kv_heads \times head_dim \times seq_len \times bytes_per_param$$

LLaMA 3 70B at 8k context in BF16: ~26 GB per request.

Context extension methods: YaRN (rescale RoPE frequencies), ALiBi (natural extrapolation), Sliding

window attention (Mistral — $O(W)$ memory instead of $O(n^2)$).

25. Flash Attention

Naive attention: $O(K^2)$ matrix = $n \times n$. 8k context $\rightarrow$ 256MB per head per layer. Bandwidth bottleneck.

FlashAttention (Dao et al., 2022): compute attention in tiles without materializing full $n \times n$ matrix. Fuse softmax + matmul into single GPU kernel. $O(n)$ memory, 2-4 $\times$ speedup.

- **FA-2:** better parallelization, native GQA. 2 $\times$ faster than FA-1.
- **FA-3:** H100-specific, async ops, FP8, ~75% H100 utilization.

26. Model Architectures Timeline

Year	Model	Org	Params	Key Contribution
2017	Transformer	Google	—	Self-attention replaces RNN
2018	BERT	Google	340M	Bidirectional encoder, MLM pretraining
2019	GPT-2	OpenAI	1.5B	Decoder-only, CLM at scale
2020	GPT-3	OpenAI	175B	Few-shot emergent abilities
2022	ChatGPT	OpenAI	~175B	SFT + RLHF alignment
2023	LLaMA	Meta	7-65B	Open weights, efficient
2023	Mistral 7B	Mistral	7B	GQA + sliding window, beats LLaMA 13B
2023	LLaMA 2	Meta	7-70B	RLHF aligned, 4k context
2024	Mixtral 8 $\times$ 7B	Mistral	47B total, 13B active	Mixture of Experts
2024	LLaMA 3	Meta	8-70B	128k vocab, strong instruct
2024	Gemma 2	Google	2-27B	GQA + logit softcapping
2024	DeepSeek-R1	DeepSeek	671B MoE	GRPO-trained reasoning

27. Mixture of Experts (MoE)

Dense model: every token uses every parameter. MoE: N expert FFN layers, router selects top-K experts per token.

$$\text{output} = \sum_{i \in \text{TopK}} \text{router_weight}(i) \times \text{Expert}_i(x)$$

Why: same parameter count $\rightarrow$ higher capacity. Same compute per token $\rightarrow$ faster inference than

equivalent dense model.

- **Mixtral 8x7B:** 8 experts, 2 active. Total ~47B, active ~13B. Performs like 34B dense.
- **DeepSeek-R1:** 671B total, ~37B active, 256 experts, top-8.
- **Challenges:** load balancing (auxiliary loss), expert routing overhead, communication across GPUs.

28. LLM Capabilities and Limitations

Capability	How
In-context learning	Few examples in prompt → model adapts without weight update
Zero-shot generalization	Instruction in prompt, model follows
Tool use / Function calling	Generate structured JSON, API routes to actual tools
RAG	Retrieve external text, inject into context
Code execution	Generate code → execute → feed result back
Agents	LLM as reasoning core, calls tools, observes, loops
Structured output	Constrained decoding (grammar-based) → always valid JSON

Limitations: arithmetic (poor number tokenization), real-time info (training cutoff), persistent memory (context window only), hallucination (confident but wrong).

Mitigations: code interpreter for math, RAG for knowledge, vector DB for memory, verifiers for correctness.

PART II: Database Management Systems (DBMS)

1.1 Why Not File Systems?

- Data redundancy (same data in multiple files)
- Inconsistency (update one file, forget another)
- No concurrent access control
- No security, backup, or querying

DBMS = software managing structured data. Handles storage, retrieval, concurrency, integrity, security.

RDBMS = Relational DBMS. Data stored in tables (relations) linked via keys. Examples: PostgreSQL, MySQL, Oracle, SQLite, SQL Server.

1.2 Key Terminology

Term	Meaning
Table (Relation)	Grid of rows and columns
Tuple (Row)	One data entry: (1, "Adith", "Class 10")
Attribute (Column)	One field: student_id, name, class
Domain	Allowed values for attribute. age: positive integers
Degree	Number of attributes (columns)
Cardinality	Number of tuples (rows)
Schema	Structure definition — blueprint. What tables, columns, types, constraints exist.
Instance	Actual data in DB at a point in time

2. Keys

Key Type	Definition	Notes
----------	------------	-------

Super Key	Any set of attributes uniquely identifying a tuple	{student_id}, {student_id, name}, {email} — all super keys if unique
Candidate Key	Minimal super key — no subset is also a super key	Can't remove any attribute and retain uniqueness
Primary Key (PK)	Chosen candidate key	NOT NULL, UNIQUE, physically indexed
Alternate Key	Candidate keys not chosen as PK	—
Foreign Key (FK)	References PK of another table	Enforces referential integrity
Composite Key	Key made of multiple attributes	(order_id, product_id) unique together
Surrogate Key	Artificial PK, no business meaning	Auto-increment integer. Preferred over natural keys.
Natural Key	PK with real-world meaning	Risky — real-world data changes

3. Relational Algebra

Operator	Symbol	Meaning
Select	σ	Filter rows: $\sigma(\text{age} > 18)(\text{Students})$
Project	π	Pick columns: $\pi(\text{name, age})(\text{Students})$
Union	$\cup$	All rows from both tables (same schema)
Intersection	$\cap$	Rows in both tables
Difference	$-$	Rows in first not in second
Cartesian Product	$\times$	Every pair from both tables
Natural Join	$\bowtie$	Join on all common attributes
Rename	ρ	Rename table/attributes

Outer Joins: LEFT (all left + matching right, NULLs for non-matches), RIGHT (opposite), FULL (all from both, NULLs where no match).

4. SQL — Complete Reference

4.1 DDL (Data Definition Language)

```

CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE,
    class_level INT CHECK (class_level IN (10, 12)),
    school_id INT REFERENCES schools(id) ON DELETE CASCADE,
    created_at TIMESTAMP DEFAULT NOW()
);

ALTER TABLE students ADD COLUMN phone VARCHAR(15);
ALTER TABLE students DROP COLUMN phone;
DROP TABLE students;           -- delete table
TRUNCATE TABLE students;       -- delete all rows, keep structure

```

4.2 DML (Data Manipulation Language)

```

-- Insert
INSERT INTO students (name, email, class_level) VALUES ('Adith', 'a@b.com', 10);

-- Update
UPDATE students SET class_level = 12 WHERE student_id = 5;

-- Delete
DELETE FROM students WHERE student_id = 5;

-- Select
SELECT name, email FROM students WHERE class_level = 10 ORDER BY name ASC LIMIT 10;

```

4.3 Advanced Queries

```

-- Joins
SELECT s.name, sc.school_name
FROM students s
INNER JOIN schools sc ON s.school_id = sc.id;

SELECT s.name, sc.school_name
FROM students s
LEFT JOIN schools sc ON s.school_id = sc.id; -- includes unassigned students

-- Aggregates + GROUP BY + HAVING
SELECT class_level, COUNT(*), AVG(score), MAX(score)
FROM students
GROUP BY class_level
HAVING COUNT(*) > 50; -- HAVING filters AFTER GROUP BY; WHERE filters BEFORE

-- Subquery
SELECT name FROM students
WHERE school_id IN (SELECT id FROM schools WHERE city = 'Mumbai');

```

```
-- Window functions
SELECT name, score,
       RANK() OVER (PARTITION BY class_level ORDER BY score DESC) as rank,
       LAG(score) OVER (ORDER BY created_at) as prev_score
FROM students;

-- CTE (Common Table Expression)
WITH top_students AS (
  SELECT * FROM students WHERE score > 90
)
SELECT school_id, AVG(score) FROM top_students GROUP BY school_id;
```

4.4 Transactions (TCL)

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;    -- make permanent
-- OR
ROLLBACK;  -- undo everything since BEGIN

SAVEPOINT sp1;
ROLLBACK TO sp1;
```

5. ACID Properties

Property	Meaning	Example
Atomicity	All or nothing	Transfer debits A, credits B — either both happen or neither
Consistency	DB moves between valid states	Constraints always satisfied before and after transaction
Isolation	Concurrent transactions don't interfere	Each sees consistent snapshot as if running serially
Durability	Committed data persists after crash	Written to disk via WAL (Write-Ahead Log)

5.1 Isolation Levels and Real Problems

Problem	Description
Dirty Read	Read uncommitted data from another transaction that may rollback
Non-Repeatable Read	Same row read twice gives different values (another committed update between reads)

Phantom Read	Re-execute query gets different rows (another transaction inserted/deleted)
--------------	---

Isolation Level	Dirty Read	Non-Repeatable	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

PostgreSQL default: READ COMMITTED. MySQL InnoDB: REPEATABLE READ. Higher isolation = more locking = less concurrency.

5.2 Concurrency Control

2PL (Two-Phase Locking): Phase 1 (Growing) acquire locks only. Phase 2 (Shrinking) release locks only. Guarantees serializability but can deadlock.

MVCC (Multi-Version Concurrency Control): readers never block writers, writers never block readers. Each transaction sees DB snapshot at start time. PostgreSQL uses this. Much better concurrency.

Deadlock: T1 holds A waits for B, T2 holds B waits for A. DBMS detects cycle → kills one transaction.

6. Normalization — Complete Guide

6.1 Functional Dependencies (FD)

$X \rightarrow Y$ means knowing X uniquely determines Y. If two rows have same X, they must have same Y.

`student_id → name ✓ (each ID has one name) name → student_id ✗ (multiple students can share a name)`

Closure of X (X⁺): all attributes determinable from X. If X⁺ = all attributes, X is a super key.

6.2 First Normal Form (1NF)

Rule: every cell contains atomic (indivisible) value. No multi-valued attributes. No repeating groups.

Violation:

```
student_id | subjects
1          | "Math, Science, English"  <-- not atomic
```

Fix: one row per subject, or separate junction table.

6.3 Second Normal Form (2NF)

Rule: in 1NF AND no partial dependency — every non-key attribute depends on the WHOLE primary key, not part of it. Only relevant for composite PKs.

Violation:

```
Table: Enrollment(student_id, subject_id, grade, student_name, subject_name)
```

```
PK = (student_id, subject_id)
```

```
student_name → depends on student_id alone ✗ PARTIAL DEPENDENCY
```

```
subject_name → depends on subject_id alone ✗ PARTIAL DEPENDENCY
```

Fix: split into Student(student_id, student_name), Subject(subject_id, subject_name), Enrollment(student_id, subject_id, grade).

6.4 Third Normal Form (3NF)

Rule: in 2NF AND no transitive dependency — non-key attribute must not depend on another non-key attribute.

Violation:

```
Student(student_id, name, zip_code, city, state)
```

```
zip_code → city ✗ TRANSITIVE (city depends on zip, not directly on student_id)
```

```
zip_code → state ✗ TRANSITIVE
```

Fix: Student(student_id, name, zip_code) + ZipCode(zip_code, city, state).

Formal 3NF definition: for every FD $X \rightarrow Y$, either X is a super key OR Y is a prime attribute (part of some candidate key).

6.5 Boyce-Codd Normal Form (BCNF)

Rule: for every non-trivial FD $X \rightarrow Y$, X must be a super key. No exceptions (stricter than 3NF).

Violation:

```
CourseTeacher(student, course, teacher)
```

```
FDs: (student, course) → teacher [PK]
```

```
teacher → course [teacher teaches one course]
```

```
teacher is NOT a super key → BCNF violated!
```

Fix: TeacherCourse(teacher, course) + StudentTeacher(student, teacher).

BCNF vs 3NF tradeoff: BCNF eliminates all redundancy but may not preserve all FDs. 3NF always preserves FDs. In practice: aim for BCNF, accept 3NF when BCNF causes FD loss.

6.6 Fourth Normal Form (4NF)

Multi-valued dependency (MVD): $X \twoheadrightarrow Y$ means for each X, set of Y values is independent of other attributes.

Violation:

```
Employee(emp_id, skill, language)
emp_id  $\twoheadrightarrow$  skill      (multiple skills, independent of language)
emp_id  $\twoheadrightarrow$  language (multiple languages, independent of skill)
Result: every skill  $\times$  language pair row – massive redundancy!
```

Fix: EmployeeSkill(emp_id, skill) + EmployeeLanguage(emp_id, language)

6.7 Normalization Quick Reference

Normal Form	Eliminates	Requires
1NF	Multi-valued / non-atomic attributes	Atomic values, PK
2NF	Partial dependencies (on part of composite PK)	1NF
3NF	Transitive dependencies (non-key $\rightarrow$ non-key)	2NF
BCNF	All FD anomalies (stricter 3NF)	3NF, accept FD loss
4NF	Multi-valued dependencies	BCNF
5NF	Join dependencies	4NF (rarely needed)

7. ER Diagrams

Symbol	Meaning
Rectangle	Entity (Student, School, Subject)
Oval	Attribute. Double oval = multi-valued. Dashed = derived.
Diamond	Relationship (Enrolls, Teaches)
Double rectangle	Weak entity (depends on another entity)
Single line	1:1 relationship
Arrow on one side	1:N relationship
Arrows both sides	M:N relationship $\rightarrow$ junction table

8. Indexing

Without index: full table scan = $O(n)$. 10M rows → scan every row.

Index Type	Structure	Good For	Bad For
B+ Tree	Balanced tree, linked leaf nodes	Equality, range, ORDER BY — $O(\log n)$	High-cardinality write-heavy
Hash	Hash function → bucket	Equality only — $O(1)$	Range queries (useless)
Composite	Multi-column B+ tree	Left-prefix queries	Queries on non-leftmost column only
Covering	Index contains all needed columns	Query served from index only	Wide indexes increase write overhead

When NOT to index: columns rarely queried, very low selectivity (e.g. boolean flags), small tables, write-heavy tables.

EXPLAIN ANALYZE: shows query execution plan. Look for Seq Scan (bad on large tables) vs Index Scan (good).

9. Transactions — Deeper

```
-- Idempotency pattern
INSERT INTO students (student_id, name)
VALUES (1, 'Adith')
ON CONFLICT (student_id) DO UPDATE SET name = EXCLUDED.name;

-- Advisory lock (application-level)
SELECT pg_advisory_lock(12345); -- acquire
SELECT pg_advisory_unlock(12345); -- release
```

10. NoSQL — When and Why

Type	Examples	Data Model	Use Case
Document	MongoDB, Firestore	JSON documents	Flexible schema, nested data
Key-Value	Redis, DynamoDB	Hash map	Sessions, caching, leaderboards
Wide-column	Cassandra, HBase	Dynamic columns per row	Time-series, IoT, high write throughput
Graph	Neo4j	Nodes + edges	Social networks, recommendations

CAP Theorem: Consistency, Availability, Partition tolerance — can guarantee at most 2 of 3 simultaneously. NoSQL often chooses Availability + Partition tolerance, giving up strong consistency.

(BASE: Basically Available, Soft state, Eventually consistent).

[illegible]

PART III: Computer Networks

1. Network Fundamentals

1.1 Network Types by Geography

Type	Scale	Examples
PAN	~10m	Bluetooth devices
LAN	Building/campus	Ethernet, WiFi
MAN	City	Cable TV, metro fiber
WAN	Countries/continents	Internet, leased lines

1.2 Network Topologies

Topology	Description	Failure Mode
Bus	All on single cable	One break = whole network down
Star	All connect to central switch	Switch failure = total failure
Ring	Devices in loop	Used in legacy Token Ring
Mesh	Every device connected to every other	Very fault tolerant, expensive
Tree	Hierarchical buses	Used in corporate networks

2. The OSI Model — All 7 Layers

Mnemonic (top to bottom): "All People Seem To Need Data Processing"

Application, Presentation, Session, Transport, Network, Data Link, Physical

#	Layer	PDU	Protocols / Devices	Key Function
7	Application	Message	HTTP, DNS, FTP, SMTP, SSH, DHCP, SNMP	User-facing services
6	Presentation	Data	SSL/TLS, JPEG, ASCII, MPEG	Format, encrypt, compress

5	Session	Data	RPC, NetBIOS, SIP, PPTP	Session establish/manage/terminate
4	Transport	Segment/ Datagram	TCP, UDP	End-to-end delivery, ports
3	Network	Packet	IP, ICMP, OSPF, BGP, RIP	Routing, logical addressing
2	Data Link	Frame	Ethernet, WiFi (802.11), ARP	MAC addressing, framing, same-LAN
1	Physical	Bit	Copper, Fiber, Radio, Hub, Repeater	Raw bits on medium

2.1 Layer 7 — Application

Interface between network and user application. Defines protocols apps use.

Protocol	Port	Purpose
HTTP	80 TCP	Web browsing (unencrypted)
HTTPS	443 TCP	Web browsing (TLS encrypted)
FTP	20/21 TCP	File transfer
SSH	22 TCP	Secure remote shell
SMTP	25, 587 TCP	Send email
IMAP	143, 993 TCP	Receive email (sync)
POP3	110, 995 TCP	Receive email (download+delete)
DNS	53 UDP/TCP	Domain name resolution
DHCP	67/68 UDP	Auto-assign IP addresses
SNMP	161/162 UDP	Network device management
NTP	123 UDP	Time synchronization

2.2 Layer 6 — Presentation

Translation, encryption, compression between application data and network format.

- **Translation:** UTF-8 ↔ UTF-16, ASCII ↔ EBCDIC
- **Encryption:** SSL/TLS (though modern TLS spans L5/L6/L7)
- **Compression:** JPEG, MPEG, GIF

2.3 Layer 5 — Session

Establishes, manages, terminates sessions between applications.

- **Session establishment:** authenticate, authorize, open session
- **Checkpointing:** resume large file transfer after network drop — no full restart needed
- **Protocols:** RPC (Remote Procedure Call), NetBIOS, SIP (VoIP setup/teardown), PPTP (VPN)

2.4 Layer 4 — Transport

TCP (Transmission Control Protocol)

- Connection-oriented — 3-way handshake
- Reliable — guaranteed delivery, ordered, no duplicates
- Flow control — receiver controls send rate (sliding window)
- Congestion control — slows down when network congested

3-way handshake:

```
Client → Server: SYN    (seq=x)
Server → Client: SYN-ACK (seq=y, ack=x+1)
Client → Server: ACK    (ack=y+1)
Connection established.
```

4-way termination:

```
Client → Server: FIN
Server → Client: ACK
Server → Client: FIN
Client → Server: ACK
Client waits TIME_WAIT (2×MSL)
```

TCP Congestion Control:

- Slow start: cwnd=1 MSS, double every RTT until threshold
- Congestion avoidance: increase 1 MSS per RTT after threshold
- Loss detected: threshold = cwnd/2, restart slow start

UDP (User Datagram Protocol)

- Connectionless — no handshake, just send
- Unreliable — no guarantee of delivery, order, deduplication
- Fast — 8-byte header vs TCP's 20 bytes
- Use cases: DNS, video/audio streaming, gaming, DHCP, NTP, QUIC/HTTP3

2.5 Layer 3 — Network

Logical addressing (IP) and routing across networks.

- **IP:** connectionless, best effort delivery

- **ICMP:** error reporting. ping = ICMP Echo. traceroute = ICMP TTL exceeded.
- **ARP:** maps IP → MAC address within same network
- **TTL:** decremented at each router, packet dropped at TTL=0 (prevents infinite loops)

Routing Protocols:

Protocol	Type	Metric	Used In
RIP	Distance-vector	Hop count (max 15)	Small networks (legacy)
OSPF	Link-state (Dijkstra)	Cost (bandwidth)	Enterprise LANs
BGP	Path-vector	Policy-based	Internet backbone between ISPs

2.6 Layer 2 — Data Link

Node-to-node transfer on same network segment. MAC addressing, framing, error detection.

MAC Address: 48-bit hardware address. AA:BB:CC:DD:EE:FF. First 3 bytes = OUI (manufacturer). Unique per NIC.

Error detection methods:

- Parity bit — detects odd number of bit errors
- **CRC (Cyclic Redundancy Check)** — polynomial division, very reliable detection. Standard in Ethernet.
- Checksum — used in IP/TCP/UDP headers

MAC Access Protocols:

Protocol	Used In	How
CSMA/CD	Wired Ethernet	Sense → Transmit → Detect collision → Jam + backoff
CSMA/CA	WiFi (802.11)	Sense → Wait DIFS → Random backoff → Transmit → Wait ACK

ARP process:

1. Device knows target IP but not MAC
2. Broadcast: "Who has IP 192.168.1.5?"
3. Target replies with its MAC
4. Requester caches in ARP table

ARP Poisoning: attacker sends fake ARP replies → traffic redirected through attacker (MITM attack).

2.7 Layer 1 — Physical

Raw bit transmission. Electrical signals, light pulses, radio waves.

Medium	Type	Speed / Range	Use
Twisted Pair UTP Cat5e	Guided copper	100 Mbps	Ethernet LAN

Twisted Pair UTP Cat6	Guided copper	1 Gbps	Ethernet LAN
Cat6a	Guided copper	10 Gbps	High-speed LAN
Coaxial	Guided copper	—	Cable TV, early Ethernet
Single-mode fiber	Guided light	Terabits, km range	WAN backbone
Multi-mode fiber	Guided light	High, 100s meters	Data centers
WiFi 2.4GHz	Unguided radio	Up to 600 Mbps (n)	Long range, more interference
WiFi 5GHz	Unguided radio	Up to 3.5 Gbps (ac)	Short range, less interference

Shannon's Capacity:

$$C = B \times \log_2(1 + S/N)$$

C = max capacity, B = bandwidth (Hz), S/N = signal-to-noise ratio. Physical limit — no encoding can exceed this.

2.8 Encapsulation

Data travels down layers; each layer adds its header. Going up on receiver: each layer strips its header.

```
App data
→ [L4 header | App data] (segment)
→ [L3 header | L4 header | data] (packet)
→ [L2 header | L3 hdr | L4 hdr | data | L2 FCS] (frame)
→ bits on wire
```

3. TCP/IP Model vs OSI

TCP/IP Layer	OSI Equivalent
Application	Application + Presentation + Session (L7+L6+L5)
Transport	Transport (L4)
Internet	Network (L3)
Network Access	Data Link + Physical (L2+L1)

4. IPv4 Addressing

32-bit address. Written as 4 octets in dotted decimal. Total: $2^{32} = \sim 4.3$ billion addresses.

192.168.10.25 = 11000000.10101000.00001010.00011001

IP address = Network ID + Host ID. Subnet mask defines the boundary.

Class	Range	Default Mask	Networks	Hosts/ Network	Use
A	1.0.0.0 – 126.255.255.255	255.0.0.0 (/8)	126	16,777,214	Large orgs, ISPs. Private: 10.x.x.x
B	128.0.0.0 – 191.255.255.255	255.255.0.0 (/16)	16,384	65,534	Medium orgs. Private: 172.16-31.x.x
C	192.0.0.0 – 223.255.255.255	255.255.255.0 (/24)	2,097,152	254	Small orgs. Private: 192.168.x.x
D	224.0.0.0 – 239.255.255.255	N/A	N/A	N/A	Multicast only. OSPF, IPTV, video conf.
E	240.0.0.0 – 255.255.255.255	N/A	N/A	N/A	Reserved experimental. Not in production.

5.1 Special Addresses

Address	Meaning
0.0.0.0	This host on this network (unknown source)
127.0.0.1	Loopback (localhost) — test own TCP/IP stack
255.255.255.255	Limited broadcast (this network)
x.x.x.0	Network address (host bits all 0)
x.x.x.255	Directed broadcast for that network (host bits all 1)
169.254.x.x	APIPA — auto-assigned when DHCP fails

5.2 Private IP Ranges (RFC 1918)

Class	Range	CIDR
A	10.0.0.0 – 10.255.255.255	/8
B	172.16.0.0 – 172.31.255.255	/12

3. Block size: $2^6 = 64$

Subnet	Network Address	First Host	Last Host	Broadcast
1	192.168.1.0/26	192.168.1.1	192.168.1.62	192.168.1.63
2	192.168.1.64/26	192.168.1.65	192.168.1.126	192.168.1.127
3	192.168.1.128/26	192.168.1.129	192.168.1.190	192.168.1.191
4	192.168.1.192/26	192.168.1.193	192.168.1.254	192.168.1.255

Hosts per subnet: $2^6 - 2 = 62$.

6.5 Find Subnet of an IP — Method

Which subnet does 192.168.1.100 belong to in /26?

AND the IP with subnet mask:

```
192.168.1.100    = 11000000.10101000.00000001.01100100
255.255.255.192 = 11111111.11111111.11111111.11000000
AND               = 11000000.10101000.00000001.01000000
                  = 192.168.1.64
```

Belongs to subnet 192.168.1.64/26. Broadcast = 192.168.1.127.

6.6 VLSM (Variable Length Subnet Masking)

Different subnets with different sizes based on actual need. Starting block: 192.168.1.0/24

Department	Hosts Needed	Subnet Size	CIDR	Range
Engineering	50	/26 (62 hosts)	192.168.1.0/26	.0 – .63
Sales	25	/27 (30 hosts)	192.168.1.64/27	.64 – .95
HR	10	/28 (14 hosts)	192.168.1.96/28	.96 – .111
Link	2	/30 (2 hosts)	192.168.1.112/30	.112 – .115

6.7 Route Aggregation (Supernetting)

Combine multiple networks into one summary route.

```
192.168.0.0/24 + 192.168.1.0/24 + 192.168.2.0/24 + 192.168.3.0/24
Common bits: first 22 bits
Summary: 192.168.0.0/22 (covers .0 through .3.255)
```

Router advertises one route instead of four. Internet runs on route aggregation.

7. IPv6

128-bit addresses. $2^{128} = 3.4 \times 10^{38}$ addresses.

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Shortened: 2001:db8:85a3::8a2e:370:7334

Shortening rules: drop leading zeros in group. Replace one run of all-zero groups with :: (only once).

Address	Meaning
::1	Loopback (equiv to 127.0.0.1)
fe80::/10	Link-local (auto-configured, not routable)
ff00::/8	Multicast

IPv6 advantages: no broadcast (only multicast), mandatory IPSec, SLAAC (stateless auto-config), no fragmentation at routers, larger but simpler fixed 40-byte header.

8. NAT (Network Address Translation)

Private IPs not routable on internet. NAT at router translates.

1. Inside host 192.168.1.5:3456 → 8.8.8.8:53
2. Router replaces source with public IP:port: 203.0.113.1:40000
3. Stores mapping in NAT table
4. Reply comes to 203.0.113.1:40000
5. Router translates back to 192.168.1.5:3456

NAT Type	Mapping	Use
Static NAT	1 private ↔ 1 public	Servers needing fixed public IP
Dynamic NAT	Pool of public IPs, dynamic assignment	Organizations with IP pool
PAT (NAT Overload)	Many private → 1 public (port-based)	Home routers — standard

9. DNS — Domain Name System

Internet's phone book. Translates human-readable names to IP addresses.

9.1 Resolution Process for www.google.com

1. Browser cache → OS cache → /etc/hosts check

2. Ask Recursive Resolver (ISP or 8.8.8.8)

3. Resolver asks Root Nameserver → "Ask .com NS at x.x.x.x"

4. Resolver asks .com NS → "Ask google.com NS at y.y.y.y"

5. Resolver asks Google's authoritative NS → "It's 142.250.67.68"

6. Resolver caches (per TTL), returns to client

2.2 DNS Records

Record	Purpose	Example
A	IPv4 address	google.com → 142.250.67.68
AAAA	IPv6 address	google.com → 2404:6800:
CNAME	Alias to another name	www.google.com → google.com
MX	Mail server	google.com → smtp.google.com
NS	Authoritative nameserver	google.com → ns1.google.com
TXT	Arbitrary text	SPF, domain verification
PTR	Reverse lookup IP → name	142.250.67.68 → google.com
SOA	Zone metadata	Serial, refresh, retry times

10.1 HTTP Methods

Method	Purpose	Idempotent	Has Body
GET	Retrieve resource	Yes	No
POST	Create resource	No	Yes
PUT	Replace resource	Yes	Yes
PATCH	Partial update	No	Yes
DELETE	Delete resource	Yes	No
HEAD	GET but no response body	Yes	No
OPTIONS	Query supported methods	Yes	No

10.2 Status Codes

Range	Category	Key Codes
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

10.3 HTTP Versions

Version	Key Feature	Problem Solved
HTTP/1.0	One request per connection	—
HTTP/1.1	Persistent connections, pipelining	Connection overhead
HTTP/2	Binary framing, multiplexing, HPACK header compression, server push	Head-of-line blocking per connection
HTTP/3	QUIC (UDP-based), built-in TLS 1.3, 0-RTT reconnect	TCP head-of-line blocking

10.4 Transport Layer Security (TLS)

1. Client sends: supported ciphers, random nonce, DH public key
2. Server sends: chosen cipher, certificate, DH public key, encrypted finished
3. Client verifies cert (CA chain), derives session key from DH exchange
4. Both have symmetric key — all further traffic encrypted

One round trip (1-RTT). 0-RTT possible for resumed sessions.

11. Networking Devices

Device	OSI Layer	Function
Hub	L1	Broadcasts bits to all ports. All ports = one collision domain. Obsolete.
Switch	L2	Forwards frames by MAC. Learns MAC table. Full-duplex. Each port = own collision domain
Router	L3	Routes packets between networks using IP. Connects LAN to WAN.

Bridge	L2	Connects two LAN segments, filters by MAC.
Gateway	L4-L7	Protocol conversion. Default gateway = your subnet's router.
Load Balancer	L4/L7	Distributes traffic across servers.
Firewall	L3-L7	Traffic filtering by rules.
Proxy	L7	Forward: client → internet. Reverse: internet → backend
Repeater	L1	Amplifies signal to extend range.
Modem	L1-L2	Digital ↔ analog conversion (DSL/cable).
Access Point	L2	WiFi bridge to wired LAN.

12. Key Ports Reference

Port	Protocol	Service
20	TCP	FTP Data
21	TCP	FTP Control
22	TCP	SSH
23	TCP	Telnet (insecure, avoid)
25	TCP	SMTP
53	TCP/UDP	DNS
67/68	UDP	DHCP server/client
80	TCP	HTTP
110	TCP	POP3
143	TCP	IMAP
161/162	UDP	SNMP
443	TCP	HTTPS
587	TCP	SMTP (submission)
993	TCP	IMAPS
995	TCP	POP3S
3306	TCP	MySQL
5432	TCP	PostgreSQL

6379	TCP	Redis
27017	TCP	MongoDB

13. Network Security

Attack	Type	Description	Defense
Packet Sniffing	Passive	Capture traffic on shared medium	TLS encryption
MITM	Active	Intercept + modify communication	TLS cert pinning, HSTS
ARP Poisoning	Active	Fake ARP replies → redirect traffic	Dynamic ARP Inspection, VLANs
DNS Poisoning	Active	Inject false DNS records into resolver cache	DNSSEC
DDoS	Active	Flood target with traffic. SYN flood, UDP flood.	Rate limiting, scrubbing centers
IP Spoofing	Active	Forge source IP	Ingress filtering (BCP38)
Replay	Active	Capture + resend valid auth packets	Nonces, timestamps, digital signatures

14. Complete Data Flow — Browser to Server

When you type `https://vidyapath.com` and press Enter:

1. DNS RESOLUTION Browser cache → OS cache → /etc/hosts → Recursive Resolver → Root NS → .com NS → vidyapath.com NS → IP address (UDP port 53) 2. TCP 3-WAY HANDSHAKE to IP:443 SYN → SYN-ACK → ACK 3. TLS 1.3 HANDSHAKE ClientHello → ServerHello + Certificate + Finished Client verifies cert, both derive session key Encrypted session established 4. HTTP GET / (encrypted inside TLS) Browser → Nginx reverse proxy → Next.js server 5. SERVER PROCESSES, RETURNS HTML
--

Browser parses → parallel DNS+TCP+TLS for CSS/JS/images
JavaScript executes, makes API calls → page rendered

EACH ROUTER ALONG THE PATH:

Strip L2 frame → Read L3 IP → Re-encapsulate in new L2 frame → Forward
(Decrement TTL, drop if 0)

Appendix: Quick Reference Cheat Sheet			
AI/ML Key Formulas			
Concept		Formula	
Attention		$\text{softmax}(QK^T / \sqrt{dk}) \times V$	
Adam update		$w \leftarrow w - \eta \times \hat{m} / (\sqrt{\hat{v}} + \epsilon)$	
Cross-Entropy		$L = -\sum y \times \log(\hat{y})$	
MSE		$L = (1/n) \sum (\hat{y} - y)^2$	
BM25		$\sum \text{IDF}(t) \times (tf \times (k1+1)) / (tf + k1 \times (1-b+b \times dl/avgdl))$	
MMR		$\lambda \times \text{relevance}(d) - (1-\lambda) \times \text{max_sim}(d, \text{selected})$	
RRF		$\sum 1/(k + \text{rank_i})$ across lists	
GRPO Advantage		$(r_i - \text{mean}(r)) / \text{std}(r)$	
LoRA		$W' = W + BA$, rank $r \ll \min(d,k)$	
DBMS Quick Reference			
Normal Form		Eliminates	
1NF		Non-atomic / multi-valued attributes	
2NF		Partial dependencies (composite PK)	
3NF		Transitive dependencies	
BCNF		All FD violations (every determinant is super key)	
4NF		Multi-valued dependencies	
ACID Property		One-line Definition	
Atomicity		All or nothing	
Consistency		Valid state to valid state	
Isolation		Concurrent = serial	
Durability		Committed = persists forever	
Networking Quick Reference			

